

Atividade Prática – Fundamentos DevOps

Problema 1 – Deploy Manual

Problema identificado

O processo de implantação (deploy) é manual, demorado e sujeito a erros.

Causas

- Ausência de automação: cada deploy exige que um responsável execute manualmente uma sequência de comandos e configurações.
- Dependência de atividades manuais, como cópia de arquivos, reinício de serviços e ajustes de configuração feitos "na mão".
- Falta de padronização no processo, fazendo com que cada deploy seja conduzido de forma diferente, dependendo de quem o executa.

Como o DevOps resolveria

Automatizando o processo de implantação por meio de pipelines de CI/CD, reduzindo a intervenção humana e garantindo que todo deploy siga exatamente os mesmos passos, testados e versionados.

Ferramenta ou prática utilizada

- CI/CD
- Jenkins, GitHub Actions ou GitLab CI/CD
- Scripts de automação (ex.: shell scripts, Ansible playbooks)

Benefícios esperados

- Redução do tempo de deploy (de horas para minutos).
- Menos erros humanos no processo de implantação.
- Maior confiabilidade e repetibilidade do processo.
- Atualizações mais frequentes, com menor risco.

Problema 2 – Falta de Comunicação

Problema identificado

Falta de comunicação entre desenvolvimento e infraestrutura.

Causas

- Equipes trabalhando de forma isolada, cada uma com suas próprias prioridades e cronogramas (silos organizacionais).
- Ausência de processos colaborativos, o que significa que problemas de infraestrutura só são descobertos tarde, já em produção.
- Exemplo prático: o time de desenvolvimento entrega uma aplicação sem informar os requisitos de infraestrutura, gerando incompatibilidades no momento do deploy.

Como o DevOps resolveria

Promovendo colaboração constante entre as equipes durante todo o ciclo de desenvolvimento, com reuniões curtas e frequentes e visibilidade compartilhada sobre o andamento do projeto.

Ferramenta ou prática utilizada

- Cultura DevOps

- Reuniões de alinhamento (dailies, retrospectivas)
- Jira, Azure DevOps ou Slack

Benefícios esperados

- Menos falhas na implantação.
- Melhor integração entre equipes.
- Maior produtividade e menos retrabalho.

Problema 3 – Integração de Código

Problema identificado

Conflitos frequentes na integração do código.

Causas

- Desenvolvedores demoram para enviar alterações, acumulando mudanças grandes e difíceis de mesclar.
- Integrações pouco frequentes, muitas vezes feitas apenas uma vez por semana ou ao final da sprint.
- Exemplo prático: dois desenvolvedores alteram o mesmo módulo por dias sem sincronizar, gerando um merge conflituoso e demorado de resolver.

Como o DevOps resolveria

Aplicando Integração Contínua (CI), incentivando commits frequentes e pequenos, integrados automaticamente ao repositório principal e validados por build/testes a cada envio.

Ferramenta ou prática utilizada

- Git
- GitHub
- GitLab
- Jenkins

Benefícios esperados

- Menos conflitos de merge.
- Correção rápida de erros, pois são detectados cedo.
- Código mais estável e integrado continuamente.

Problema 4 – Testes Manuais

Problema identificado

Os testes são demorados e podem deixar passar erros.

Causas

- Falta de automação nas rotinas de teste, exigindo execução manual repetitiva.
- Dependência de testes manuais, o que é lento e sujeito a falhas humanas de observação.
- Exemplo prático: um bug de regressão simples só é percebido pelo usuário final, pois o teste manual não cobriu aquele cenário por falta de tempo.

Como o DevOps resolveria

Automatizando testes durante o pipeline de desenvolvimento, garantindo que cada alteração de código seja validada automaticamente antes de avançar para produção.

Ferramenta ou prática utilizada

- Testes automatizados
- Selenium
- JUnit
- Cypress

Benefícios esperados

- Maior qualidade do software entregue.
- Menor tempo de validação a cada ciclo.
- Redução de falhas em produção.

Problema 5 – Ambiente Diferente

Problema identificado

Diferenças entre ambiente de desenvolvimento e produção.

Causas

- Configuração manual de servidores, o que gera divergências difíceis de rastrear.
- Falta de padronização entre os ambientes de desenvolvimento, homologação e produção.
- Exemplo prático: a aplicação funciona no ambiente do desenvolvedor, mas falha em produção devido a uma versão diferente de biblioteca instalada manualmente (o clássico "na minha máquina funciona").

Como o DevOps resolveria

Utilizando ambientes padronizados e infraestrutura como código (IaC), garantindo que development, homologação e produção sejam idênticos em configuração.

Ferramenta ou prática utilizada

- Docker
- Kubernetes
- Terraform
- Ansible

Benefícios esperados

- Ambientes iguais em todas as etapas.
- Menos erros na produção causados por divergência de ambiente.
- Implantações previsíveis e reproduzíveis.

Relação entre os Problemas

Embora tenham sido apresentados separadamente, os cinco problemas estão interligados e, em geral, se retroalimentam. A falta de comunicação (Problema 2) favorece integrações de código pouco frequentes (Problema 3), que por sua vez aumentam o risco de erros não detectados por testes manuais (Problema 4). Esses erros, somados a ambientes diferentes entre desenvolvimento e produção (Problema 5), tornam o deploy manual (Problema 1) ainda mais arriscado e demorado. Ou seja: resolver apenas um desses pontos isoladamente tem impacto limitado — é a adoção conjunta da cultura e das práticas DevOps que rompe esse ciclo, automatizando o fluxo de ponta a ponta e aproximando as equipes.

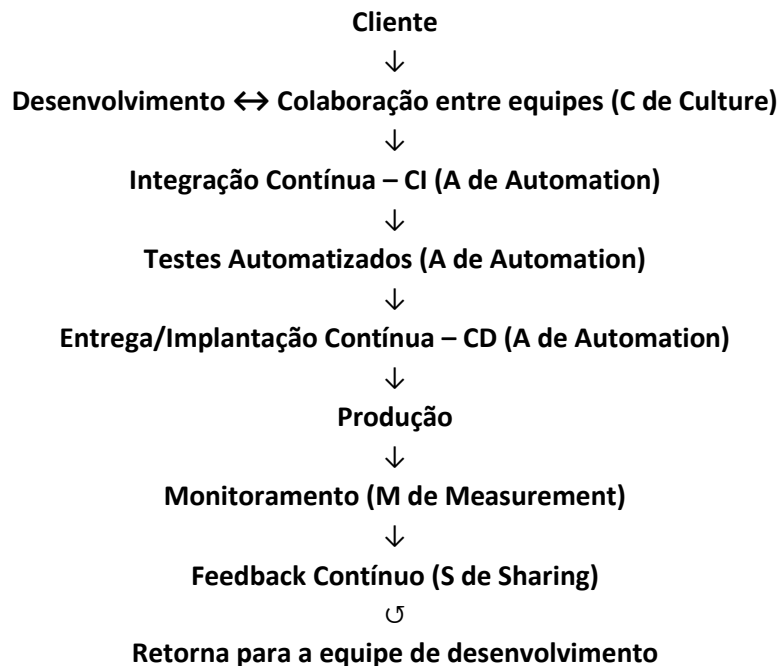
Desafio Final

Problema escolhido: Problema 1 – Deploy Manual

Antes do DevOps



Depois do DevOps (utilizando os pilares CAMS)



Pilares CAMS utilizados

- C – Culture (Cultura): colaboração entre desenvolvimento, operações e demais equipes, com comunicação constante e responsabilidade compartilhada.
- A – Automation (Automação): CI, CD e testes automatizados eliminando etapas manuais e repetitivas.
- M – Measurement (Medição): monitoramento contínuo do sistema e métricas de desempenho e estabilidade.
- S – Sharing (Compartilhamento): feedback contínuo e compartilhamento de conhecimento entre as equipes.

Métricas para medir o sucesso da transformação

Para comprovar que a adoção do DevOps trouxe resultados reais, é importante acompanhar indicadores objetivos ao longo do tempo:

Métrica	O que mede	Objetivo com DevOps
Lead Time for Changes	Tempo entre o commit do código e sua chegada em produção	Reduzir de dias para horas/minutos
Deployment Frequency	Com que frequência novos deploys são realizados	Aumentar (deploys diários ou por demanda)
MTTR (Mean Time to Recovery)	Tempo médio para recuperar o sistema após uma falha	Reduzir para minutos
Change Failure Rate	Percentual de deploys que causam falha em produção	Manter abaixo de 15%